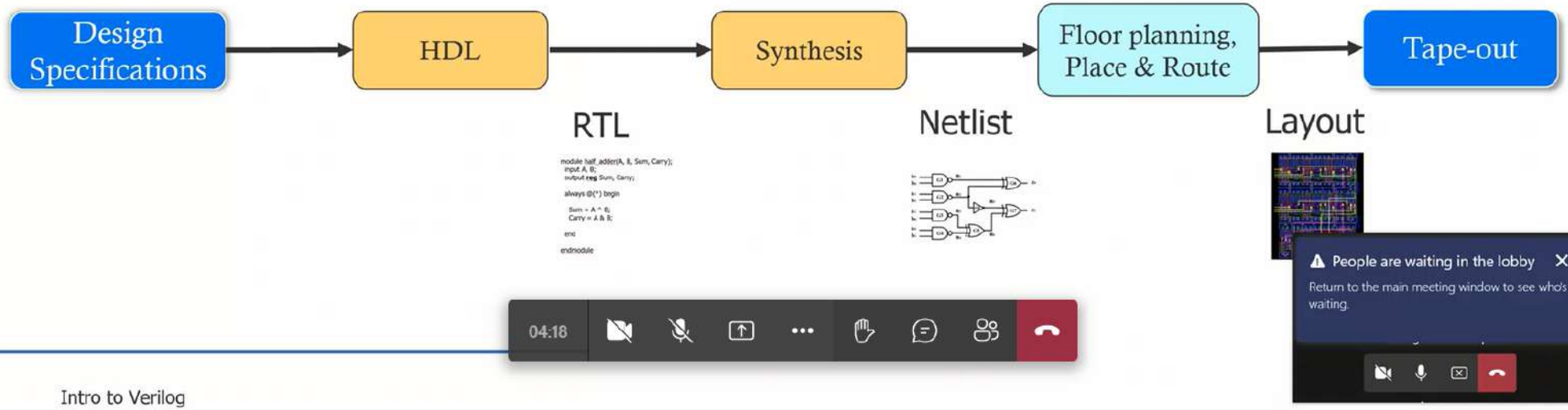
Presenting... Give control Stop presenting

HDL: Hardware Description Language

- HDL can be used to describe the behavior of digital designs/systems
- Originally HDL were used to document the behavior of digital systems
- Extended to simulate the behavior of a digital system
- Verilog HDL is a general-purpose hardware description language
  - Easy to learn and easy to use
  - Similar in syntax to the C programming language

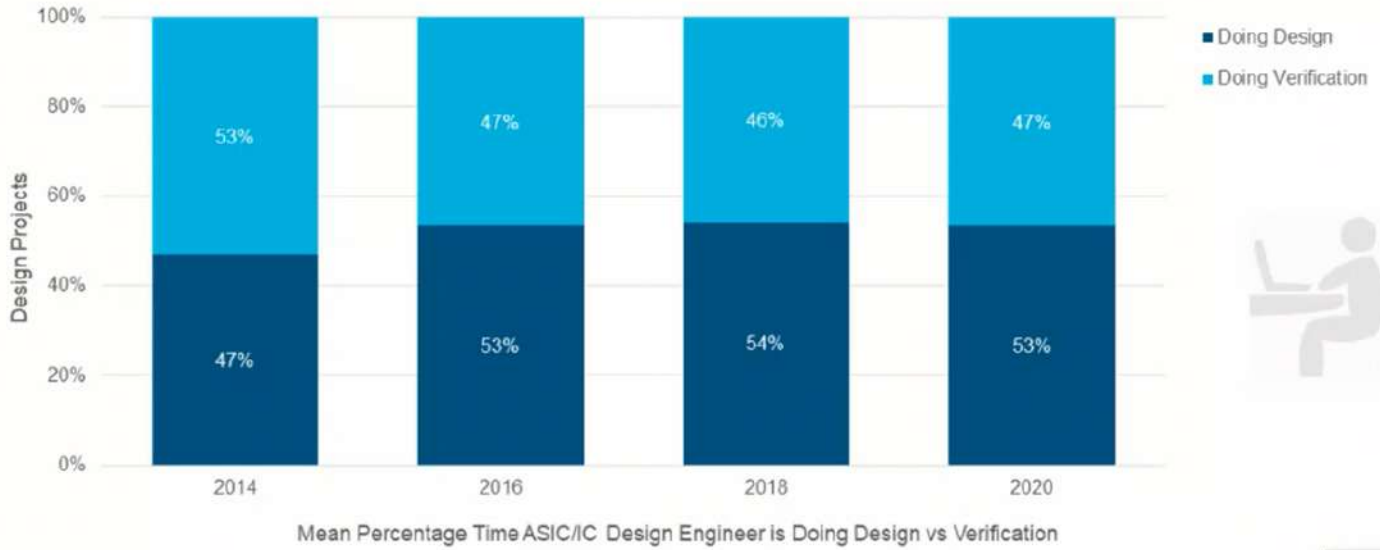


Elsaid, Kareem (DI SW ICS MNA RD QA QSIM)

Presenting... Give control Stop presenting

Where design Engineers spend their time

- Verification engineers are not the only project stakeholders involved in the verification process
- Design engineers spend a significant amount of their time in verification too



04:33

People are waiting in the lobby
Return to the main meeting window to see who's waiting.

Elsaid, Kareem (DI SW ICS MNA RD QA QSIM)

Presenting... Give control Stop presenting

Verilog Constructs

- The Verilog file has suffix *.v
- Verilog is case sensitive
- Verilog assignment, definition, declaration is terminated with semicolon (;)
- Single line Comment in Verilog is preceded a double slash "//"
- Block comment starts with /* and ends with */

```
/* block comments
/*
shall not */
be nested   */
```

```
// Single line comment in Verilog

/* This block is
commented
in Verilog */
```

05:09

Siemens FV Diploma: Verilo... 24:27
AF
Using Custom Setup

Elsaid, Kareem (DI SW ICS MNA RD QA QSIM)

Presenting... Give control Stop presenting

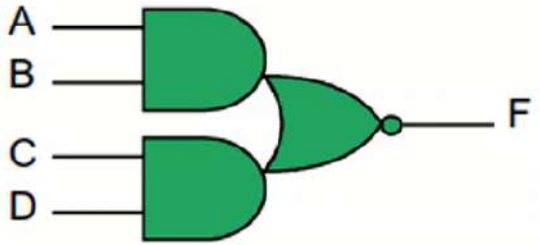
Modules and ports

```
// An and-or-invert gate

module AOI (A,B,C,D,F);
input A, B, C, D ;
output F;

// The function of the AOI module is described here…

endmodule
```



05:20

Siemens FV Diploma: Verilo... 24:38
AF
Using Custom Setup

Elsaid, Kareem (DI SW ICS MNA RD QA QSIM)

Presenting... Give control Stop presenting

Numeric Constants

- Data values should reflect the values that appear on wires

Value	Meaning
0	Logic zero, "low"
1	Logic one, "high"
Z or ?	High impedance (tri-state buses)
X	Unknown value (simulation)

- "X" is used by the simulators when the wires are not initialized to a known value
- Constants are unsigned by default
- Legal base specifications

Decimal	d	123 // default: decimal radix 'd123 // 'd = decimal radix
Hexadecimal	h	'h7B // 'h = hex radix
Octal	o	'o173 // 'o = octal radix
Binary	b	'b111_1011 // 'b = binary radix, "_" are ignored

05:39

Siemens FV Diploma: Verilo... 24:55
AF
Using Custom Setup

Elsaid, Kareem (DI SW ICS MNA RD QA QSIM)

Presenting... Give control Stop presenting

Sized Constant Numbers

- Constant will be extended with 0's to the left in all cases except the following case
  - High-order bit is X or Z, the extended bits will be X or Z too

```
16'd5       // 16-bit constant 'b0000_0000_0000_0101
4'b1001     // is a 4-bit binary number
5'd 3       //  is a 5-bit decimal number
3'b01x      // is a 3-bit number with the least significant bit unknown
12'h11      // is a 12-bit constant `b0000_0001_0001
12'hx1      // is a 12-bit constant `bXXXX_XXXX_0001
```

size

Number system

value

05:53

Siemens FV Diploma: Verilo... 25:10
AF
Using Custom Setup

Elsaid, Kareem (DI SW ICS MNA RD QA QSIM)

Presenting... Give control Stop presenting

Operators

- Bitwise Operators
  - Performs bit by bit evaluation between two operands
  - 2'b01 & 2'b11 = {0&1, 1&1} = 2'b01
  - ~(3'b011) = 3'b100
- Logical Operators
  - Returns one-bit 1 or 0 (true or false)
  - !(2'b01) = 0
  - "==" tests logical equality (tests for 1 and 0, all other will result in x)
- Reduction Operator
  - Evaluate bit by bit of a vector
  - &(4'b0111) = {0&1&1&1} = 0
- Arithmetic Operators
  - Used to perform arithmetic operations
- Relational Operator
  - Compares 2 operands and returns 1 or 0

Reduction

&a	AND
~&a	NAND
|a	OR
~|a	NOR
^a	XOR
~^a ^~a	XNOR

Bitwise

~a	NOT
a & b	AND
a | b	OR
a ^ b	XOR
a ~^ b a ^~ b	XNOR

Logical

!a	NOT
a && b	AND
a || b	OR
a == b a != b	[in]equality returns x when x or z in bits. Else returns 0 or 1

Arithmetic Operators

+ (addition)
- (subtraction)
* (multiplication)
/ (division)
% (modulus)

Relational Operators

< (less than)
<= (less than or equal to)
> (greater than)
>= (greater than or equal to)
(equal to)
(not equal to)

06:05

Siemens FV Diploma: Verilo... 25:23
AF
Using Custom Setup

Elsaid, Kareem (DI SW ICS MNA RD QA QSIM)

Presenting... Give control Stop presenting

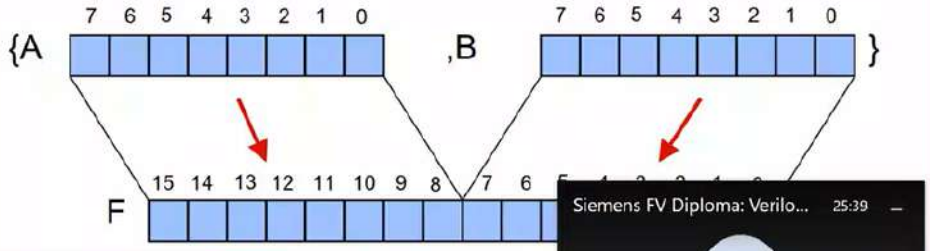
Nets: reg & wire

- physical wires(nets) are represented in Verilog using two datatypes
  - reg
  - wire

```
wire a, b, z;                  // three 1-bit wires
wire [31:0] memdata;           // a 34-bit bus
reg [7:0] b1, b2, b3, b4;      // four 8-bit buses
```

- Note that [7:0] means that 7 is the most significant bit & 0 is the least significant bit
- Concatenation using curly brackets can be used

```
wire [7:0] A, B;
reg [15:0] F;
F = {A, B}
```



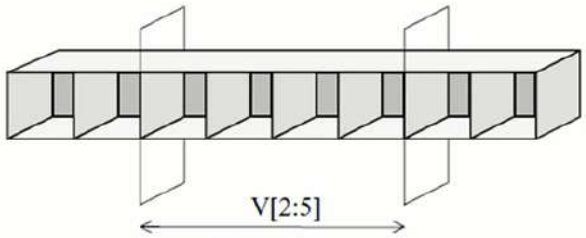
06:23
Siemens FV Diploma: Verilo... 25:39
AF
Using Custom Setup

Elsaid, Kareem (DI SW ICS MNA RD QA QSIM)
KE
Elsaid, Kareem (DI SW ICS ...

Presenting... Give control Stop presenting

Part select

```
reg [0:7] V;
reg [3:0] W;
```




```
W = V[2:5];
```

W: V[2] V[3] V[4] V[5]

```
W[1:0] = 2'b11;
```

	3	2	1	0
W:	V[2]	V[3]	1	1

06:50

Siemens FV Diploma: Verilo... 26:08
AF
Using Custom Setup

Elsaid, Kareem (DI SW ICS MNA RD QA QSIM)

Presenting... Give control Stop presenting

Procedural Block: always

- Two type of codes in almost all HDLs
  - Dataflow
  - Behavioral/Procedural
- Behavioral/Procedural
  - Statements are executed inside procedural blocks
  - Two of the most used procedural block are *always* and *initial* (later) blocks
  - Procedural blocks are executed in parallel
  - These blocks support C-like control structure such as **if, for, while, case**
  - Statements are executed in order like C code
  - **reg** data type **must** be declared for net on the left hand side of any assignment inside the procedural blocks
- always block
  - The statements inside the always block are executed sequentially
  - The statements are only executed whenever the signals inside the sensitivity list are updated
  - always @(*) can be used to trigger the block whenever any RHS nets are updated
  - Note that always @(a, b, c) is the same as always@(*)

```
always @ (<sensitivity_list>) begin
  .
  .
  <statements executed in order>
  .
  .
end
```

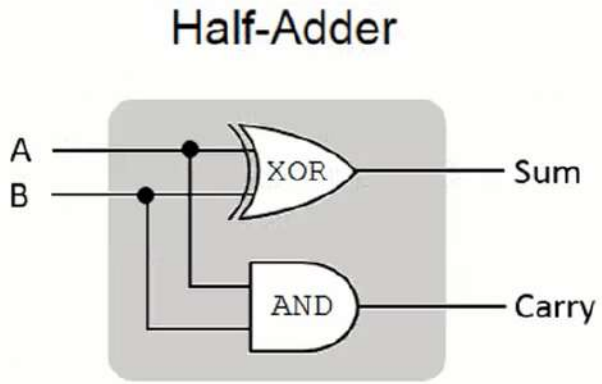
07:01

Siemens FV Diploma: Verilo... 26:18
AF
Using Custom Setup

Elsaid, Kareem (DI SW ICS MNA RD QA QSIM)

Presenting... Give control Stop presenting

Example: Half Adder




```
module half_adder(A, B, Sum, Carry);
  input A, B;
  output reg Sum, Carry;

  always @(*) begin

    Sum = A ^ B;
    Carry = A & B;

  end

endmodule
```

07:29

People are waiting in the lobby
Return to the main meeting window to see who's waiting.

Elsaid, Kareem (DI SW ICS MNA RD QA QSIM)

Presenting... Give control Stop presenting

If else

- If .. else if .. else statements execute a statement, or a block of statements depend on the result of the expression

Syntax

```
if (expression) begin
... statements ...
end
else if (expression) begin
... statements ...
end
...more else if blocks ...
else begin
... statements ...
end
```

```
If (a == b) begin
  y = 2'b01;
  out = 4'b1000;
end
else begin
  y = 2'b00;
  out = 4'b00;
end
```

```
if (alu_op == 2'b00)
  aluout = a + b;
else if (alu_op == 2'b01)
  aluout = a - b;
else if (alu_op == 2'b10)
  aluout = a & b;
else // alu_op == 2'b11
  aluout = a | b;
```

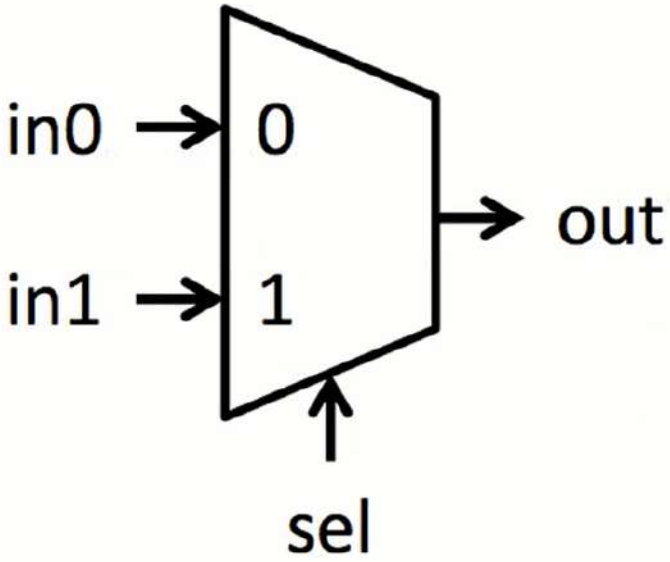
07:33

People are waiting in the lobby
Return to the main meeting window to see who's waiting.

Elsaid, Kareem (DI SW ICS MNA RD QA QSIM)

Presenting... Give control Stop presenting

Example: 2:1 MUX




```
module mux2(in0, in1, sel, out);
  input in0, in1, sel;
  output reg out;

  always @(*) begin

    if (sel == 1) begin
      out = in1;
    end
    else if (sel == 0) begin
       out = in0;
    end

  end

endmodule
```

07:53

Siemens FV Diploma: Verilo... 27:10
AF
Using Custom Setup

Elsaid, Kareem (DI SW ICS MNA RD QA QSIM)

Presenting... Give control Stop presenting

Case

- Case allow multiple branch based on comparing the expression
- Statements in the default block is executed when none of the other case choices are true
- If the default are don't care define them as 'x' or '0'

```
case ({alu_op_0, alu_op_1})
  2'b00: aluout = a + b; // if alu_op_0, alu_op_1 is 2'b00
  2'b01: aluout = a - b;
  2'b10: aluout = a & b;
  default: aluout = 1'b0;
endcase
```

Syntax

case (expression)
case_choice1:
begin
... statements ...
end
case_choice2:
begin
... statements ...
end
... more case choices blocks ...
default:
begin
... statements ...
end
endcase

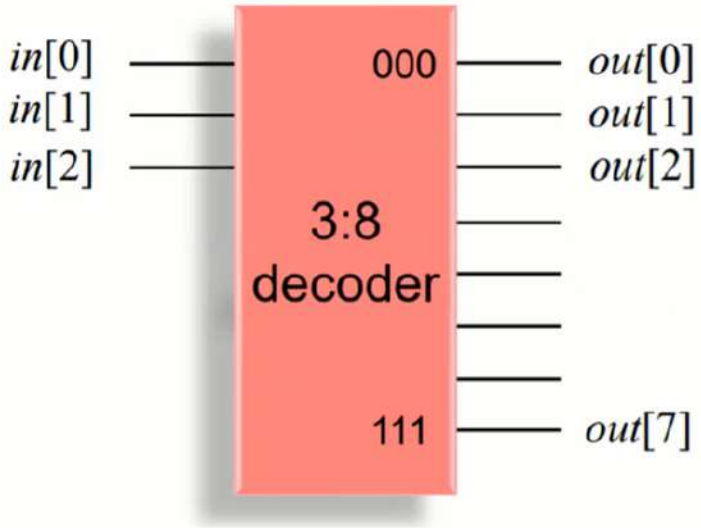
07:57

Siemens FV Diploma: Verilo... 27:13
AF
Using Custom Setup

Elsaid, Kareem (DI SW ICS MNA RD QA QSIM)

Presenting... Give control Stop presenting

Example: 3:8 Decoder




```
module decoder(in, out);
  input [2:0] in;
  output reg [7:0] out;

  always @(in) begin
    case(in)
      3'b000: out=8'b0000_0001;
      3'b001: out=8'b0000_0010;
      3'b010: out=8'b0000_0100;
      3'b011: out=8'b0000_1000;
      3'b100: out=8'b0001_0000;
      3'b101: out=8'b0010_0000;
      3'b110: out=8'b0100_0000;
      3'b111: out=8'b1000_0000;
    endcase
  end
endmodule
```

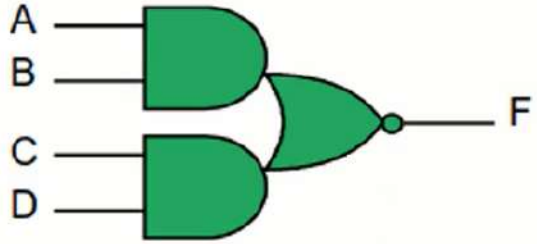
08:04

Siemens FV Diploma: Verilo... 27:21
AF
Using Custom Setup

Elsaid, Kareem (DI SW ICS MNA RD QA QSIM)

Presenting... Give control Stop presenting

Continuous assignment




```
// An and-or-invert gate
module AOI (A, B, C, D, F);
  input A, B, C, D;
  output F;

  assign F = ~((A & B) | (C & D));
endmodule
```

&	and
|	or
~	not

08:08

Siemens FV Diploma: Verilo... 27:25
AF
Using Custom Setup

Elsaid, Kareem (DI SW ICS MNA RD QA QSIM)

Presenting... Give control Stop presenting

Mux using conditional operator

```
assign out = (sel == 1)? in1 : in0;
```

/* if the sel signal == 1 then "out = in1", else "out = in0" */

08:42

Siemens FV Diploma: Verilo... 28:00
AF
Using Custom Setup

Elsaid, Kareem (DI SW ICS MNA RD QA QSIM)

Presenting... Give control Stop presenting




```
module ct1 (A, B, C1, C2, C3, F);
  input A, B, C1, C2, C3;
  output reg F;

  reg z1;

  always@(*) begin
    if (c1)
      z1 = B;
    else
      z1 = A;

    if (c2 & c3)
      F = ~z1;
    else
      F = z1;
  end
endmodule
```

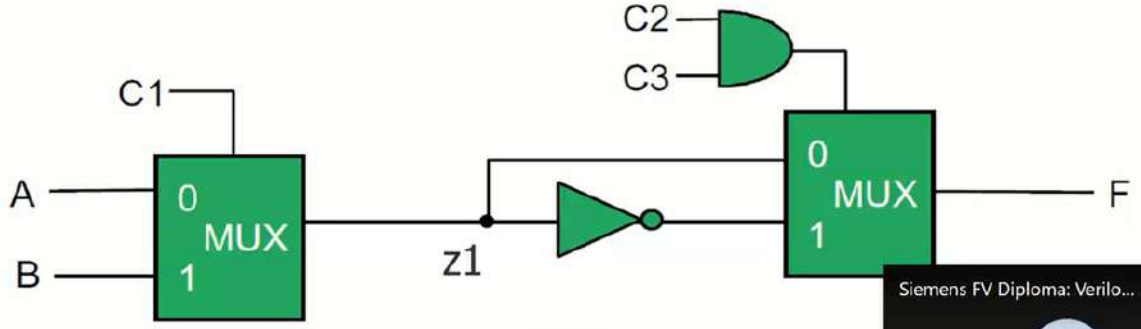
```
module mux2(in0, in1, sel, out);
  input in0, in1, sel;
  output reg out;

  always @(*) begin

    if (sel == 1) begin
      out = in1;
    end
    else if (sel == 0) begin
      out = in0;
    end

  end

endmodule
```



08:50

Siemens FV Diploma: Verilo... 28:07
AF
Using Custom Setup

Elsaid, Kareem (DI SW ICS MNA RD QA QSIM)

Presenting... Give control Stop presenting

Module Instantiations

- Module instances provide the ability to reuse any given module inside other modules
- Bigger and more complex designs are built by integrating multiple modules in a hierarchical manner
- Modules can be instantiated within other modules and ports of these instances can be connected with other signals inside the parent module
- For example, 2x1 mux can be implemented and then used multiple times inside a bigger designs instead of rewriting its functionality again
  - 4x1 MUX can be implemented using 2x1 MUX

```
<module_name>  <instance_name_1> (port_connection_list);
   mux2            m1          (.sel(sel[0]), .in0(d0), .in1(d1), .out(z1));   // MODULE PORT MAPPING BY NAME

OR
   mux2            m1          (sel[0], d0, d1, z1);                           // MODULE PORT MAPPING BY ORDER
```

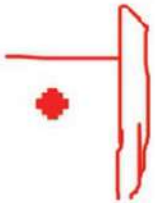
09:27

Siemens FV Diploma: Verilo... 28:44
AF
Using Custom Setup

Elsaid, Kareem (DI SW ICS MNA RD QA QSIM)

Parameterized Modules

- Suppose we have need to have a MUX with variable bit width for the data to be selected
- Parameters provides the flexibility to change word length later in any module instantiation by changing only the parameter value
  - Parameter must be declared before the module ports



Syntax for declaring parameters of a module

parameter par_1 = *value*,
par_2 = *value*, ...;
parameter [range] parm_3 = value

Syntax to override parameters when instantiating a module

module_name #(1st_parameter_values,
2nd_parm_value, ...)
instance_name(port_connection_list);

```
module mux2(in0, in1, sel, out);
 parameter W2 = 1; // width of data is by default 1 bit
 input [W2-1:0] in0, in1;
 input sel;
 output [W2-1:0] out;

 assign out = (sel == 1)? in1 : in0;

endmodule
```

01:17:32

Elsaid, Kareem (DI SW ICS MNA RD QA QSIM)

ما
محمد خالد محمد علي ا...